

Sistemas Operativos

Práctico 3: Gestión de la memoria y memoria virtual

1. Mostrar la evolución de un heap de tamaño de 1MB gestionado con un sistema buddy con bloques de tamaño entre 2^5 y 2^{10} con la secuencia de las siguientes operaciones:
 - a. `ptr1 = malloc(20)`
 - b. `ptr2 = malloc(50)`
 - c. `ptr3 = malloc(1010)`
 - d. `free(ptr1)`
 - e. `malloc(64)`

En un sistema buddy con bloques de tamaño entre 2^5 y 2^{10} , un heap de tamaño 1MB (1024 KB), puede ser de tamaño 1024KB (osea el bloque completo 1MB), 512KB, 256 KB, 128 KB, 64KB y 32KB. Son 6 tamaños distintos. [1024KB, 512KB, ..., 32KB] hasta completar el 1MB.

- a. Al realizar `ptr1 = malloc(20)`, el heap de 1MB se divide en 2:
 - 512KB y 512KB, como este tamaño sigue siendo muy mayor, divide uno de los bloques. [512KB | 512KB]
 - 256KB y 256KB, como este tamaño sigue siendo mayor, vuelve a dividir uno de estos bloques. [256KB | 256KB | 512KB]
 - 128KB y 128KB, como este tamaño sigue siendo mayor, divide uno de los bloques de 256 KB. [128KB | 128KB | 256KB | 512KB]
 - 64KB y 64KB, como este tamaño sigue siendo mayor, divide uno de los bloques de 64KB. [64KB | 64KB | 128KB | 256KB | 512KB]
 - El bloque de 64 KB lo dividió en 32 KB y 32 KB, el cual es el límite, si bien es mayor al 20 KB que pide, este es el máximo que se puede dividir. Por ende se lo asigna. [32KB (ptr1) | 32KB | 64KB | 128KB | 256KB | 512KB]

Así quedamos con el heap así:

- [32KB (ptr1) | 32KB | 64KB | 128KB | 256KB | 512KB] con el bloque de 32 KB asignado.

- b. Para la siguiente operación: `ptr2 = malloc(50)`. Hay un bloque de 32KB libre, pero es muy pequeño, con lo cual se queda con el de 64KB.

El heap queda así:

- Libres: [32KB (ptr1) | 32KB | 64KB(ptr2) | 128KB | 256KB | 512KB]

- c. La operación: `ptr3 = malloc(1010)`, pide 1010 KB, pero el HEAP ya está fragmentado y no tenemos ningún bloque de 1024 KB, con lo cual falla.
 - d. `free(ptr1)`, libera el bloque de 32KB, con lo cual el heap queda de la siguiente manera: [32KB | 32KB | 64KB(ptr2) | 128KB | 256KB | 512 KB], al liberarse el de 32KB y estar libre el otro bloque que dividimos anteriormente también de 32KB, se acoplan formando un bloque de 64KB. [64KB| 64KB(ptr2) | 128KB | 256KB | 512KB].
 - e. `malloc(64)`, pide un bloque de 64 KB, el cual tenemos lugar gracias a que los 2 bloques de 32 anteriormente divididos están libres.
El heap queda así: [64KB (ocupado)| 64KB(ptr2) | 128KB | 256KB | 512KB].
2. Hacer un programa de usuario en Linux o MS-Windows que permite analizar en qué direcciones lógicas se encuentran las áreas de código, datos, código, heap y stack. Hacerlo también en xv6.

El resultado es el siguiente:

Dirección de código (función): 0x634b1a08b189

Dirección de datos (variable global): 0x634b1a08e010

Dirección de heap (malloc): 0x634b288442a0

Dirección de stack (variable local): 0x7ffd1edb9c3c

PREGUNTARLE AL PROFE PORQUE EL STACK NO HACE REFERENCIA AL MISMO LUGAR DE LA TABLA DE PAGINA ROOT? 634b

3. En un SO en una arquitectura de 32 bits con paginado. Determine la cantidad de entradas de tablas de páginas (*pte*) y cantidad de memoria total requeridas para completar un espacio de direcciones de 4GB.
 - a. **Con páginas de 4KB de tamaño.**

En una arquitectura de 32 bits se tienen: 2^{32} bytes = 4 GB, espacio de direcciones de memoria.

Se nos dice que el tamaño de página es de 4 KB. Convertimos 4 KB a bytes:

- $4\text{KB} = 4 \times 1024 \text{ bytes} = 4096 \text{ bytes}$

Dividimos el total del espacio de direcciones por el tamaño de cada página:

- El total del espacio de dirección es: $4294967296 \text{ bytes} / 4096 \text{ bytes de el tamaño de cada página}$, obtenemos $4294967296 / 4096 = 1.048.576$ de entradas

Esto significa que la tabla de páginas debe tener **1.048.576 entradas de Página (PTE)**, para poder cubrir la totalidad del espacio virtual.

La cantidad de memoria total requerida para completar un espacio de dirección de 4GB (osea el tamaño de la estructura)

- **Cada entrada** de la tabla mapea una página de 4 KB.
- Dado que hay 1,048,576 páginas, necesitas esa cantidad de entradas.
- Y si cada entrada ocupa 4 bytes, la tabla completa ocupa:
 $1.048.576 * 4 \text{ bytes} = 4.194304 \text{ bytes} / 1024 \text{ bytes} = 4096 \text{ KB} / 1024 \text{ KB} = 4 \text{ MB}$.

Es decir, para poder gestionar un espacio de 4 GB con paginación de 4 KB, la memoria que se usa solo para la **tabla de páginas es de 4 MB**.

b. Con páginas de 4MB de tamaño.

Como dijimos se tienen: $2^{32} \text{ bytes} = 4 \text{ GB}$, espacio de direcciones de memoria.

Se nos dice que el tamaño de página es de 4 MB. Convertimos 4 GB (de espacio de direcciones de memoria) a MB:

- $4 \text{ GB} = 4 \times 1024 \text{ MB} = 4096 \text{ MB}$

Dividimos el total del espacio de direcciones por el tamaño de cada página:

- El total del espacio de dirección es: $4096 \text{ MB} / 4 \text{ MB}$ de el tamaño de cada página, obtenemos $4096 / 4 =$ de entradas

Esto significa que la tabla de páginas debe tener **1024 entradas de Página** (PTE), para poder cubrir la totalidad del espacio virtual.

La cantidad de memoria total requerida para completar un espacio de dirección de 4GB (osea el tamaño de la estructura):

- **Cada entrada** de la tabla mapea una página de 4 MB de memoria física.
- Dado que hay 1024 páginas, necesitas esa cantidad de entradas.

Memoria total requerida para la tabla de páginas: Multiplicamos el número de entradas por el tamaño de cada entrada y si cada entrada ocupa 4 bytes, la tabla completa ocupa:

- $1024 * 4 \text{ bytes} = 4096 \text{ bytes} / 1024 \text{ bytes} = 4 \text{ KB}$.

Es decir, para poder gestionar un espacio de 4 GB con paginación de 4 MB, la memoria que se usa solo para la tabla de páginas es de 4 KB.

4. Es deseable que una tabla de páginas se represente usando bloques del tamaño de una página. Proponga una representación para un espacio de direcciones de 32 bits con páginas de 4MB y de 4KB. Indique cómo debería interpretarse una dirección virtual y el mapping a direcciones físicas.

HECHO EN EL CUADERNILLO.

PREGUNTAR SI PARA PÁGINAS DE 4MB, hace falta representarlo como arboles.

5. Dos procesos comparten un área de memoria de 2 páginas. En el primer proceso esa área de memoria se mapea a partir de la dirección 0xA000. En el segundo desde 0xF000. Mostrar con un diagrama el layout de memoria de cada proceso y las tablas de páginas.

HECHO EN EL CUADERNILLO.

6. Determinar los espacios de memoria (incluyendo swap) que están usando los procesos de su sistema. Ayuda: Ver los comandos `top`, `free`, `smem` y `swapon`.
 - `top` → monitoreo en tiempo real de procesos y recursos.
 - `free` → visión rápida de memoria RAM y swap.
 - `smem` → análisis detallado de consumo de memoria por proceso (incluyendo memoria compartida).
 - `swapon` → gestión de los espacios de intercambio (swap): listar, activar y desactivar

Al ejecutar `free -h`:

	total	used	free	shared	buff/cache	available
Mem:	3,5Gi	2,6Gi	271Mi	531Mi	1,4Gi	924Mi
Swap:	4,1Gi	118 Mi	4,0Gi			

- 3,5GB Es la ram instalada (estoy usando mi pc vieja).
- Está usando 2.6GB de ram, lo que incluye procesos activos, mas lo que el kernel reserva para buffers y cache.
- Libre tiene 271 MB.
- Shared (531 MiB), es la cantidad de memoria compartida entre procesos (zonas de intercambio como pipes, etc.).
- Buff/Cache (1,4 GiB): La memoria del kernel esta usando para acelerar el proceso de acceso al disco como buffers de E/S.
- Available (924 MiB): Estimación de cuánta RAM puede usar tu sistema para nuevas aplicaciones sin recurrir a swap.

```

          total  used  free
Swap:  4,1Gi 118 Mi 4,0Gi
Al ejecutar swapon:
NAME          TYPE  SIZE  USED  PRIO
/swapfile     file  4,1G  152,1M  -2

```

Total swap (4,1 GiB): Espacio en disco reservado para el paginado.

Used swap (≈118 MiB) **(NO ENTIENDO EXACTAMENTE QUE ES ESTO).**

Free swap (4,0 GiB): espacio sin usar.

Nose que es PRIO del swap

7. En un kernel (como en GNU-Linux), sus direcciones lógicas comienzan a partir de la dirección base (sea KERNBASE). El espacio de direcciones de los procesos comienzan desde cero. Proponer una función o expresión para que el kernel determine el contenido de una dirección virtual va de un proceso a una dirección del kernel y viceversa.

???????????

8. Compilar y ejecutar (en Linux) el archivo mmapfile.c dado.
 - a. El archivo creado (mmapfile.txt) ocupa poco más de 4096 bytes. Porqué el acceso a data + 7000 no causa un segmentation fault?

El acceso a data + 7000 no causa un segmentation fault, ya que al llamar mmap le asigna un tamaño de PGSIZE * 2 (que son 8192 bytes), osea el sistema asigna dos páginas completas de memoria. Ya que el mapeo se realiza en base a páginas, y cada página tiene un tamaño de 4096 bytes.

- b. Modificar el programa para hacer el mapeo del archivo para lectura y escritura y que las modificaciones queden almacenadas en el archivo automáticamente al liberar el mapping. ¿Cómo hace esto el SO?

En este caso tenemos que usar mmap(..., MAP_SHARED), el kernel instancia una Área de la memoria virtual que hace referencia al archivo en el disco que va apuntar al archivo en el page cache (básicamente una copia del archivo). Cada página de ese rango se resuelve bajo demanda: al acceder, se produce un page fault que carga la página desde el disco al page cache. Y si cierro el archivo o elimino el mapeo del archivo, los cambios se ven reflejados en el disco.

Cómo hace esto el sistema operativo:

- Al hacer mmap, el kernel instancia una área de la memoria virtual para que apunte al page cache del kernel. Cada página de ese rango se resuelve bajo demanda, osea, que al acceder, produce un page fault (si aún no fue cargada bit V = 0) que carga la

pagina desde el disco a la page.

- Al abrirlo con PROT_WRITE, podemos escribir esas páginas. Si el programa escribe en una página mapeada, el hardware marca el bit d (dirty) de esa página.
- Al hacer munmap(): el kernel libera el espacio de la memoria virtual que apunta al archivo y recorre las páginas sucias (dirty) osea las que fueron modificadas y escribe las páginas modificadas a el archivo (en el disco). Al haber realizado una memoria virtual, las páginas modificadas se van a escribir correctamente en la posición que van de el archivo.

EL PROFE DIJO:

Al mmap hay que decirle que permita escribir, le tengo que decir que con PROT_WRITE, para cuando se libere el map, las páginas que yo modifique que se vayan a escribir a disco automáticamente.

Fijarse al final con que tamaño quedó el archivo.

- c. Verificar el tamaño del archivo luego que finalice el programa.

En mi caso el archivo una vez escrito con mmap y haber cerrado el mapeo, si bien los cambios son reflejados correctamente en la primera página, en la segunda no. Ya que cuando busque en LINUX un mmap no extiende el tamaño del fichero a menos que se amplíe (es decir que en el disco sea de 8kb en este caso).

9. Compilar y ejecutar los programas dados shmwriter.c y shmreader.c (en ese orden) para poder observar cómo funciona la API shared memory (IPC de UNIX).
 - a. Correr nuevamente el programa shredder y ver qué ocurre.
- Lo que ocurre es que una vez que yo escribí en la memoria compartida, al ejecutar shredder, este solo muestra lo que hay en ella y luego la elimina con

```
shmctl(shmid, IPC_RMID, NULL);
```

Por ende, una vez que vuelva a correr el programa, esta parte de la memoria había sido eliminada, por ende despues al obtener la memoria virtual de la dirección con esa key, no muestra nada, ya que si vuelves a ejecutar cualquiera de los dos programas, shmget con la misma clave creará un nuevo segmento (vacío).

- b. Comentar la línea que destruye la shm en shmreader.c. Correr el programa shmwriter y luego correr varias veces shmreader.
- Si comentamos la línea shmctl(shmid, IPC_RMID, NULL); en shmreader.c, entonces el segmento no se elimina.
 - Entonces cuando volvemos a ejecutar shmreader (ejecutar antes shmwriter y luego varias veces shmreader), siempre tenemos el mismo segmento de memoria compartida.

- Por tanto, la dirección virtual que imprime shmat es la misma en cada ejecución, porque el kernel vuelve a ligar el mismo bloque al mismo espacio de direcciones
 - c. Modificar ambos programas para que escriban y lean datos en la dirección lógica *str+500*.